A Mixed-Pruning Based Framework for Embedded Convolutional Neural Network Acceleration

Xuepeng Chang, Huihui Pan, Weiyang Lin, and Huijun Gao, *Fellow, IEEE*

***Abstract*— Convolutional neural networks (CNN) have been proved to be an effective method in the field of artificial intelligence (AI), and large-scale deploying CNN to embedded devices, no doubt, will greatly promote the development and application of AI into the practical industry. However, mainly due to the space-time complexity of CNN, computing power, memory bandwidth and flexibility are performance bottlenecks. In this paper, a framework containing model compression and hardware acceleration is proposed to solve the above problems. This framework consists of a mixed pruning method, data storage optimization for efficient memory utilization and an accelerator for mapping CNN on field programmable gate array (FPGA). The mixed pruning method is used to compress the model, and data bit-width is reduced to 8-bit by data quantization. Accelerator based on FPGA makes it flexible, configurable and efficient for CNN implementation. The model compression is evaluated on NVIDIA RTX2080Ti, and the results illustrate that the VGG16 is compressed by 30× and the fully convolutional network (FCN) is compressed by 11× within 1% accuracy loss. The compressed model is deployed and accelerated on ZCU102, which is up to 1.7× and 24.5× better in energy efficiency compared with RTX2080Ti and Intel i7 7700.**

***Index Terms*— Convolutional neural network, model compression, hardware acceleration, FPGA.**

I. Introduction

CONVOLUTIONAL neural networks (CNN) have shown great advantages in the fields of computer vision, including object classification and image segmentation in recent years [1], [2]. After AlexNet made a major breakthrough in image recognition, many kinds of CNN have emerged, such as VGGNet, GoogleNet, and ResNet [2]–[4]. An appropriate model with enough training data will lead to good results in corresponding task, which is even better than manual participation. In general, the deeper networks or networks with more parameters have better performance. Nevertheless, large models bring high computational complexity and space complexity, and bring great challenges to the computing power and cache of the device.

As the most common industrial application fields of deep learning, intelligent cars and unmanned aerial vehicles have high requirements on the real-time hand accuracy of models [5]–[7]. Meanwhile, due to the hardware limitations of embedded devices, higher demands are put forward for the scale and energy efficiency of deep learning models. Therefore, it is of great practical significance to design an architecture for the deployment of neural network.

For embedded applications, graphics processing units (GPU), application specific integrated circuit (ASIC) and FPGA are three common devices for inference of CNN. GPUs are widely used in training and inference, and provide high bandwidth and computing power. Increasing batch size is the strategy GPUs used to accelerate CNN. Nevertheless, large batch size is not practical for real-time applications and will cause great latency [8]. The high power dissipation of GPU also limits its application in embedded systems [9]–[11]. The superiorities of ASIC are high efficiency and performance, whereas the development cycle and flexibility are the practical issues to be considered. FPGA is appropriate for inference on embedded systems since it combines flexibility, high efficiency, and low power dissipation [12]–[14].

Using limited on-chip memory and bandwidth has become the major factors limiting the performance of FPGA [15], [16], since it is nearly impossible to deploy a model with millions of parameters on a commonly used FPGA. Previous works have used compression algorithms to reduce the size of models. "Deep Compression" algorithm [17], Han *et al.*, has achieved tens of times compression of the models by weight pruning, quantization and Huffman encoding without affecting the accuracy. Reference [18] uses low-rank approximated channel pruning to accelerate CNN. Completely removing some filters is also a method of model compression, and it does not result in unstructured sparsity [19], [20]. Deep Neural Network Development Kit tool provided by Xilinx offers pruning and quantization platform for CNN inference applications, which can also reduce the model size.

Considering the power dissipation and efficiency requirement of embedded systems, such as intelligent vehicle [21]–[24], mapping CNN onto FPGA with model compression is a viable solution. Several FPGA-based CNN accelerator architectures have been investigated. Some automatic CNN accelerator generators have been proposed in prior studies, which can shorten the development cycle and are friendly to software developers, but the capabilities of the device may not be fully available for the deployment of specific networks. Other architectures are dedicated to a certain kind of network, and show nice speed-up performance [25]–[28]. The layers of CNN are instantiated and mapped onto FPGA in architectures above, and multiple layers can be combined to form a network flexibly.

Manuscript received May 25, 2020; revised November 4, 2020; accepted December 27, 2020. Date of publication January 12, 2021; date of current version March 8, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant U1964201, Grant 61803120, and Grant 61790562. This article was recommended by Associate Editor W. X. Zheng. *(Corresponding author: Huihui Pan.)*

The authors are with the Research Institute of Intelligent Control and Systems, Harbin Institute of Technology, Harbin 150001, China (e-mail: xuepengchang@gmail.com; huihuipan@hit.edu.cn; wylin@hit.edu.cn; hjgao@hit.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSI.2020.3048260.

Digital Object Identifier 10.1109/TCSI.2020.3048260

1549-8328 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.

Motivated by the above discussion, this paper proposes a framework to accelerate and deploy CNNs on FPGA, using model compression algorithm and memory optimization strategy with acceptable accuracy loss and high performance, which is suitable for running on embedded systems with low power dissipation. The contributions of this paper are summarized as follows:

1. In this paper, the CNN model is compressed before deployment, and a mixed-pruning method is proposed to compress the neural network. The mixed-pruning method uses different strategies for pruning different network structures to achieve maximum compression. Under the premise of constant precision, the parameters and operations of network can be reduced by over 50%, which provides a prerequisite for solving the memory and bandwidth bottleneck of embedded devices.

2. A data storage and access strategy for sparse models is proposed. Due to the sparsity of the compressed model, the data access and computing efficiency will be reduced if the sparse model is deployed directly. An improved CSC storage format is designed to optimize sparse matrix storage, and a row tile method is proposed to access feature map and model parameters efficiently.

3. Customized processing engines for common layers are designed to improve computing efficiency. Users can configure any number and type of processing engines to complete the accelerated deployment of neural network. Data are processed as a stream in a processing engine, and multiple processing engines make up pipelines to parallel computing, which achieves efficient resource utilization and high processing speed.

The remainder of this paper is organized as follows: In Section II, computing principles of CNN are introduced, and two main pruning methods are listed. Section III elaborates the overall framework design flow, including the proposed mixed pruning method and architecture of our accelerator. The storage policy and the acceleration method of computing module are also emphasized. The performance of the architecture and experimental analysis is presented in Section IV. The conclusions of this paper are presented in Section V.

II. Background

A. Convolutional Neural Networks

A typical CNN is composed of several network layers, in which the most common layers are convolution layer, pooling layer and FC layer. Feature maps are most often three-dimensional for visual applications, two of which represent the size of the features and the last represents different features of the input feature map. The convolution layer uses filters to extract the features of the input image and changes the dimension of the feature map. In most cases, a non-linear activation function is appended to the convolution operation. The convolution layer can be expressed as follows [2], [3], [14], [16]:

$$F_o(n) = f_{act}\left(b_n + \sum_{m=0}^{M-1}\left(W(m,n) * F_i(m)\right)\right) \qquad n = 0, 1, \ldots, N-1. \quad (1)$$

TABLE I
CHARACTERISTICS OF CMOMON CNNS

Net	Parameters (Million)	Operation (Gops)
VGG16	138.35	34.09
FCN8s	18.64	43.03
GoogleNet	6.97	3.16
ResNet-50	25.5	7.72

where $\boldsymbol{F_i(m)}$ is the mth channel of input feature map and $\boldsymbol{F_o(n)}$ is the nth channel of output feature map. $\boldsymbol{M}$ and $\boldsymbol{N}$ represent the number of channels for input and output. $\boldsymbol{W(m, n)}$ is the mth channel of nth kernel and b is the bias term. $\boldsymbol{f_{act}}$ is the activation function, commonly used are ReLU, sigmoid or tanh.

Convolution is essentially multiply-accumulate (MAC), which consumes most of the computing resources. As the network deepens and the number of kernels increases, it is compute-bound to deploy CNN on embedded devices. The FC layer operates as a classifier, classifying objects according to the extracted features. The FC operation is equivalent to matrix multiplication, despite taking up fewer logical resources, it is memory-bound since the weight of FC layer is large and non-reusable.

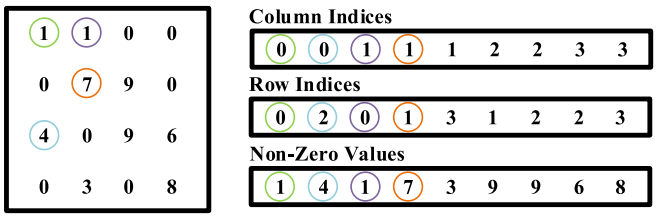
B. Neural Network Compression

The high memory usage, computation complexity and high power dissipation limit the implementation of original CNN on embedded systems. The number of parameters and operations needed for typical CNN is shown in Table I. Taking VGG16 as an example, it has more than 138 million parameters with 527MB storage spaces, among which convolution layers account for 10% and the FC layers account for the last 90%. Embedded devices do not have the capacity to process such large amounts of data. Network compression makes this possible.

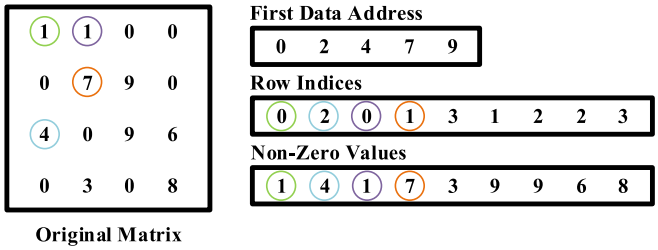
The idea of network compression is to remove parameters that contribute little to the results. Pruning is an effective mean to compress CNN by removing low-weight connections. Generally, weight pruning and filter pruning are two mainly used method of pruning. Filter pruning removes unimportant filters and corresponding feature maps directly [29], [30]. The pruned filters decrease the dimension of the feature maps, which may result in loss of accuracy [31]. The idea of weight pruning is removing nonsignificant weights whose eigenvalue is less than the threshold. The eigenvalue can be L1-norm, L2-norm or user-defined value [32]–[35]. Some elements of the weight matrix will be deleted after weight pruning, making it a sparse matrix.

There are three common sparse matrix storage formats, including coordinate (COO), compressed sparse row (CSR) and compressed sparse column (CSC) format [36]. The COO format uses row index, column index and data value to represent non-zero weights, indicating that it needs to store $3n$ numbers, as shown in Fig. 1(a), where n is the number of non-zero elements in the sparse matrix. The CSR and CSC format store the data value and column or row index, and use another array to store the address of the data in the data value array that is the first one appears on a new row or column, as shown in Fig. 1(b). Therefore, the CSR and CSC format also

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.



(a) COO Format



(b) CSC Format

Fig. 1. Sparse matrix storage.

use three arrays but only need to store $2n + m + 1$numbers of data, where m is the number of columns or rows. A new format called compressed coordinate (CCOO) [37], only uses two arrays with $2n$ numbers, one of which is the weight array storing non-zero elements, and the other stores the number of zeros between two adjacent non-zero weights.

1) Data Quantization: Quantization is a way to reduce network size by reducing the bit-width of data. Fine-tuning is needed in most cases during the training phase which requires floating-point precision. But the situation is different for inference, CNN has high tolerance to noise and the data precision falling also can be treated as a kind of noise, meaning that accurate prediction results can also be obtained with low bit-width data [38].

In this paper quantization strategy that reduces data precision is used. The data quantization is executed after pruning in order to simplify the training phase, and the data bit-width is quantified from float-32 to integer-8 since FPGA has advantages of low precision fixed-point calculation. The low precision model reduces the need for computational power and bandwidth, making the models smaller and inference efficiency higher. A suitable mapping from float-32 to integer-8 is needed to keep the accuracy, and direct mapping may result in a serious loss of accuracy. For most models, the parameter values are mainly distributed in $[-1, 1]$, therefore, it is possible to artificially narrow the representation range of float-32 to reduce the precision loss caused by bit-width drop. Inspired by [39], the mapping used in this paper can be represented by the following formulas:

$$scale = (x_{float}^{max} - x_{float}^{min}) \div (x_{int}^{max} - x_{int}^{min}) \quad (2)$$

$$zero_point = x_{int}^{max} - x_{float}^{max} \div scale \quad (3)$$

$$x_{int} = round(x_{float} \div scale + zero_point) \quad (4)$$

where x_{float}^{max} and x_{float}^{min} represents the maximum and minimum values of float-32 format, and x_{int}^{max} and x_{int}^{min} represents the maximum and minimum values of int-8 format. The $scale$ is a const represanting the mapping scale from floating number to integer, and $zero_{point}$ is used to mapping the zero point of float-32 to integer-8.

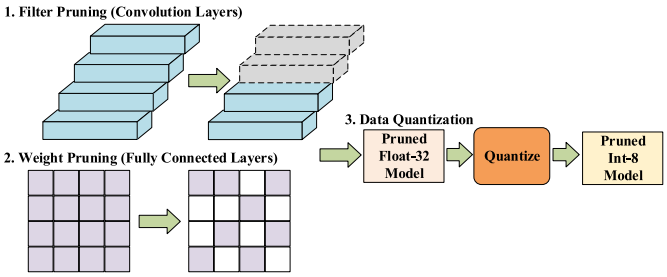


Fig. 2. The mixed pruning method.

III. Framework Design Flow

A. The Mixed Pruning Method

Compared with GPU, the inference of CNN on FPGA is limited by memory bandwidth. For maximum compression of the model to reduce memory footprint, a mixed pruning method is proposed in this paper as shown in Fig. 2. The combination of filter pruning and weight pruning can achieve better performance than the single method.

Pruning reduces the size of the model but also brings storage problems. The pruned convolution kernels change to sparse matrixes, and do not always reduce the number of parameters. Assume an $m \times m$ matrix having m^2 non-zero weights, it has $2n + m + 1$ parameters represented by CSC format, after pruning with pruning rate r, where n equals $m \times m \times (1 - r)$. Assuming that the original parameters are stored with rows in the main order, to ensure the amount of parameters drops, inequation (5) should be satisfied.

$$2(m \times m \times (1 - r)) + m + 1 < m \times m. \quad (5)$$

The result is that the pruning rate gets smaller as the m gets larger, and the minimum is 50%. For the convolution layer, pruning more than 50% of the weights could result in large accuracy loss. Therefore, weight pruning is unsuitable for the convolution layer. But FC layers have lots of useless weights that can be pruned, the pruning rate could over 90% with little accuracy loss.

On the basis of the above analyses, the mixed pruning method first applies filter pruning to the convolution layers with a relatively low rate to the minimal damage to network structures, as shown in Algorithm 1.[1] Fine-tune can recover the lost performance of the model to some extent. Then parameters in convolution layers are fixed and the whole convolution part is used as a feature extractor. For classification networks containing the FC layer, such as VGG16, most of the parameters are in FC layers. Weight pruning is used after filter pruning and prunes the FC layers with a high pruning rate, doing little harm to the accuracy. Only the filter pruning method is used for FCN since it only has the convolution layer and deconvolution layer. A preliminary tiny CNN model is obtained after filter pruning and weight pruning. On this basis, data quantification is also necessary to improve the execution efficiency of FPGA.

[1]In Function 'FixedTrain', the parameters of the convolution layer are fixed and no update is performed.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 The Mixed Pruning Algorithm

Input: the pre-pruned CNN model
RL: the filter pruning ratio (relatively low)
RH: the weight pruning ratio (relatively high)
Output: the pruned CNN model

```
foreach epoch ∈ epochs do
    Train(model)
    foreach filter ∈ model do
        filter = 0 if rank(filter) < RL
    end
    pruned model = Save(model) If Acc. Up
end
foreach epoch ∈ epochs do
    FixedTrain(pruned model)
    foreach weight ∈ FC do
        weight = 0 If rank(weight) < RH
    end
    pruned model = Save(pruned model) If Acc. Up
end
Output(pruned model)
Quantize(pruned model)
```

An efficient storage policy is presented in Section III-B2 to solve the problem of sparse data structure triggered by the mixed pruning.

B. Hardware Implementation

1) Specific Architecture: This paper presents a flexible, hardware parallelized and memory-optimized CNN accelerator architecture. Fig. 3 shows the overall design of the architecture. The images and pruned CNN model as the input of the accelerator are stored in the onboard SD card. Data are preloaded into the DDR during the inference, and data needed to be processed currently is transferred to the Zynq chip. On-chip BRAMs are divided into three parts: feature map tiles buffer, parameters buffer and the output buffer. Almost all the operations of CNN are carried out in the computing module, containing convolution processing engine (CPE) array, FC processing engine (FCPE) and data loading/writing module. The arithmetic units and control logic are deployed on programmable logic (PL) programming system (PS) respectively. The data transmission between PL and PS uses the AXI bus.

2) Storage Policy:

a) Feature map: Filter pruning removes unimportant kernels of convolution layers. Meanwhile, it indirectly reduces a portion of feature maps and lowers the burden on memory. Unfortunately, the on-chip memory of FPGA is still insufficient to fit an entire input feature map of large images. This paper proposes a memory-optimized method, using input segmentation and memory reuse to reduce the memory footprint, of which the latency is decreased by pipeline.

Xilinx FPGAs use Block Random Access Memory (BRAM) as the on-chip storage unit. Each BRAM consists of two RAMs, the capacity of which is 18Kbits. Multiple BRAMs stitching is used to store one channel of the feature map and row tile method is used to read the stored data for

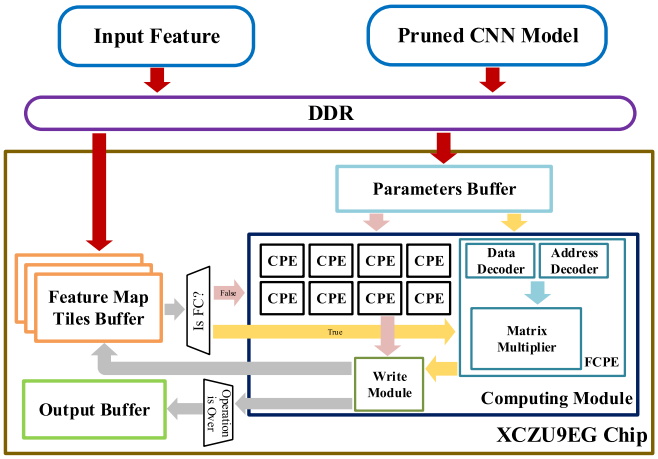


Fig. 3. The overall design architecture.

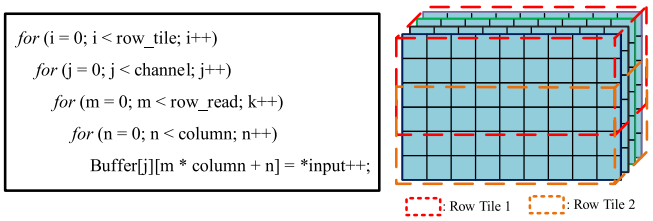


Fig. 4. Row tile method.

the fully use of memory. Take the second convolution layer of VGG16 as an example, the input feature map is 64 dimension with size 224 × 224, after pruning and quantization it remains 52 channels of 8-bit integer format data, a single channel occupies 224 × 224 × 8 bits and has already exceeded the max depth of a BRAM. Hence, each BRAM is allocated to store a channel of the feature map in time-sharing. The data of each channel is read together so that 52 feature map pixels can be accessed at one clock cycle. One BRAM can store 4500 pixels (or 20 rows) and 12 row tiles are needed to access all the pixels in a channel. The four loops in Fig. 4 are fully pipelined in FPGA and several extra iterations are needed to balance the memory usage and timing sequence required. Notice that there is some overlap between tile1 and tile2 in Fig. 4, because convolution requires a few rows in front of the new row.

b) Convolution layer weights: The parameters will not change and be exactly the same for different input images for a trained model during inference phase. In ideal conditions, carving out part of on-chip storage space is the best solution to reuse the weights and only need to read the parameters once. Like the way to store feature map, FPGA has insufficient memory, but it is possible for one convolution layer with parameters less than 18Mb. The weights can be reused by input channels during current layer processing.

c) FC layer weights: In this paper modified CSC format is applied to store the after-pruning sparse matrix. For the FC layer, the weight matrix is converted to a sparse format, and the input vectors also become a sparse vector after layers of pruning. In general, the row number of weight matrix is usually over one thousand, which is out of range for 8-bit data. So the number of zeros between two contiguous

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.

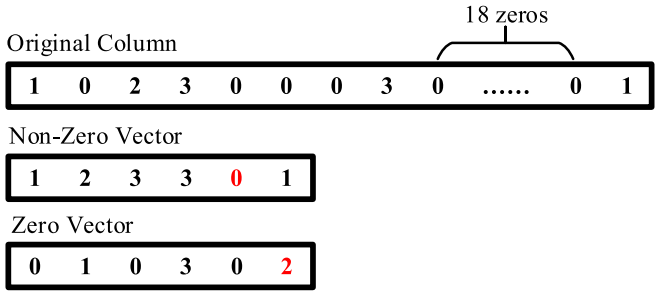


Fig. 5. Modified CSC row indices vector.

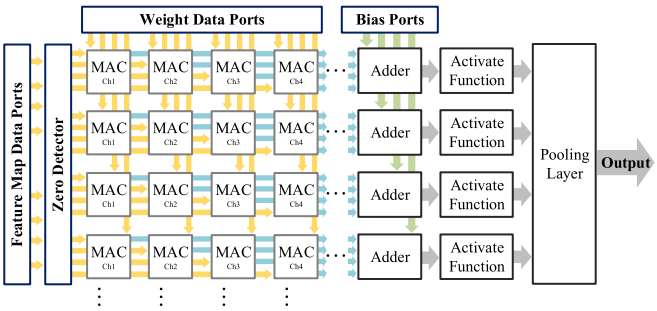


Fig. 6. Convolution processing engine.

non-zero weights are stored instead of row indices. If the zeros are more than 16, a zero value are added to the non-zero vector, as shown in Fig. 5. Compared with the CCOO format, accessing data from the specified address makes the modified CSC format more suitable for pipeline operation and more efficient for FPGA implementation. For a sparse weight matrix $\boldsymbol{W}$ with i rows and j columns, a vector $\boldsymbol{w}$ is used to store all non-zero elements of each column and a vector $\boldsymbol{z}$ to record the number of contiguous zeros. The vector $\boldsymbol{w}$ and $\boldsymbol{z}$ have equal-length and the element of $\boldsymbol{z}$ is represented in 4-bit value. In case there are more than fifteen zeros before a non-zero weight, a zero will be added to the vector $\boldsymbol{w}$ for every 15 consecutive zeros. The input feature vector has the same storage policy, but the weight matrix has the third vector $\boldsymbol{p}$ to store the pointer pointing to the first non-zero element of each column. The number of non-zero weights in jth column can be calculated by $\boldsymbol{p}_{(j+1)} - \boldsymbol{p}_j$.

3) Convolution Processing Engine: Convolution is the most frequently used computation in CNN. A configurable convolution structure will have a great effect on improving the execution efficiency. The convolution processing engine (CPE) is a common module responsible for the related calculation and optional operation (pooling or activation function) of the convolution layer, as shown in Fig. 6. Multiple CPEs can be used in parallel to speed up the network and the amounts of CPEs can be adjusted for different layers.

A CPE mainly contains three parts: MAC module, pooling module and activation function module. Every time the convolution layer starts, feature maps and weights are transferred to CPEs through the data ports. MAC module is the main operating unit of CPE, using a channel of $k \times k$ kernel and corresponding $k \times k$ part of the feature map as input, and output the multiply-accumulate result. Every time getting an output pixel, the activation function will be applied to it to add nonlinear characteristics. After one row tile of convolution operation finished, the pooling module will down-sample the output feature, mean-pooling or max-pooling can be selected. Each CPE is responsible for a row tile and multiple CPEs can implement a convolution layer.

a) MAC module: MAC can complete the convolution operations of a single-pixel of a channel. MAC modules are implemented using DSP48E, the on-chip DSP of Xilinx devices. The previous works show that exploiting internal multipliers and adders of DSPs to complete the whole convolution operation can realize the maximum utilization of on-chip resources but bring the decline of the maximum frequency, so dedicated adders are used to add the intermediate result of MAC operation.

In order to achieve maximum computational parallelism, every channel of input data should be calculated simultaneously and added using a multi-input adder. In each iteration of row tiles, the input feature data and weight data are loaded to MAC modules, the corresponding elements of every channel are computed together and each channel of row tile is processed by a group of MAC modules to decrease the delays. The MAC module that completes the operation will be released immediately and reused in the next row tile iteration, which will maximize DSP resources and improve processing efficiency.

b) Activation function module: In addition to convolution operation, CPE can also apply the activation function to the output data. Generally used activation functions are supported such as ReLU, sigmoid or tanh. Whether or which activation function to use is controlled by the PS processor. Take ReLU as an example, it is implemented by a comparator, when ReLU function is enabled, the comparator outputs the result of the comparison between the data and zero as a mask. The mask will be applied to the input of ReLU function with positive input is '1' and negative input is '0'.

c) Pooling module: The pooling layer is often used for reducing the size of the feature map by downsampling. The pooling module is an optional module of the CPE after convolution operations which consists of comparator, adder and shift register. Supposing that pooling module is enabled, the module will be launched when two rows of data are exported since the pooling layer using 2×2 windows over the feature maps. For max pooling, only comparators are used, outputs the pixel with the largest value. And for mean pooling, adders and shift registers are used to find the mean value of the data covered by the sliding window. The configurations of the pooling module is determined before run time and couldn't be changed.

d) Deconvolution module: The purpose of convolution is to obtain high dimensional feature maps with tiny size. Deconvolution is used to restore the high dimensional features and realize the pixel level classification [40]. Deconvolution is also called transposed convolution, which in previous works are converted into convolution and use the method of convolution for processing. For a deconvolution defined by kernel size k, padding p and stride s, it performs equivalently to a convolution with $k' = k$, $s' = 1$ and $p' = k - p - 1$ and adding $s - 1$ zero padding between pixels of origin input feature maps.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Data Decode and FC Operation

Input: **a**: input vector of FC layer
w: pointer to non-zero weights of FC layer
z: stores the distance between non-zero weights
p: pointer the first non-zero weights of columns
m, n: define the size of origin weight matrix
Output: **b**: output vector of FC layer

```
N = 0
j = 0
foreach j < n do
    i = 0
    k = 0
    foreach k < (p[j + 1] − p[j]) do
        i += z[k]
        b[j] += w[N + k] * a[i]
        i += 1
        k += 1
    end
    N += p[j + 1] − p[j]
    j += 1
    if UseActFun then
        b[j] = act_fun(b[j])
    end
end
```

Since deconvolution can be changed to convolution, CPEs are used to implement it on FPGA using the same way as convolution. Before processed by CPEs, data preprocessing is needed, the input feature maps should be padded when transmitted from on-chip buffers to CPEs. Because a lot of zeros exist in the feature maps, comparators are used to zero-multiply instead of DSPs.

4) FC Layer Processing Engine: As specified in the previous section, data decoder is needed since the weights of the FC layer can not be used directly with modified CSC format. For an m × n sparse weight matrix stored using three vectors $\boldsymbol{w}$, $\boldsymbol{z}$ and $\boldsymbol{p}$, and feature maps are transmitted to the FC layer in vector form, the output vector can be obtained by Algorithm 2. N is used to record the sum of non-zero elements in each column. The index of input vector $\boldsymbol{a}$ corresponding to the non-zero weight is decoded and then the output vector $\boldsymbol{b}$ will be calculated by multipliers and adders. Activation function is also available for $\boldsymbol{b}$ if necessary.

IV. Evaluation

A. Experimental Platform

1) CPU: CPU is a kind of general-purpose processor, which determines that its execution efficiency for specific problems is relatively low, acting as the baseline for performance evaluation. This paper uses intel i7 7700, and PyTorch for CPU version to deploy CNN. The power measurement of the CPU is reported by s-tui tool.

2) GPU: This paper uses NVIDIA GeForce RTX 2080Ti GPU, which is the state-of-the-art desktop GPU for deep learning training and inference. With 76TOPS and 616GB/s memory bandwidth, 2080Ti has adequate hardware resources to do deep learning related operations. Training and compressing are both performed on the GPU using PyTorch for GPU version. Command *nvidia-smi* is used to measure the power consumption of the GPU.

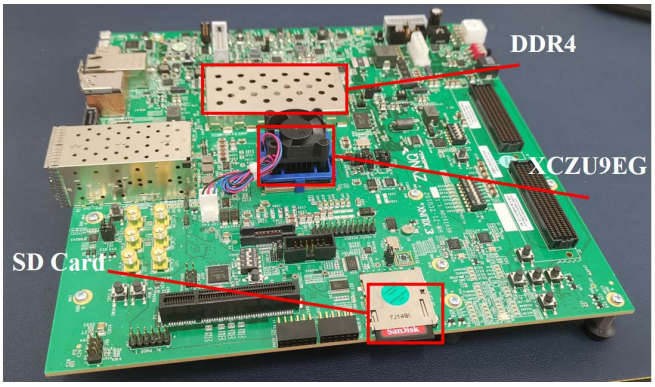


Fig. 7. ZCU102 platform.

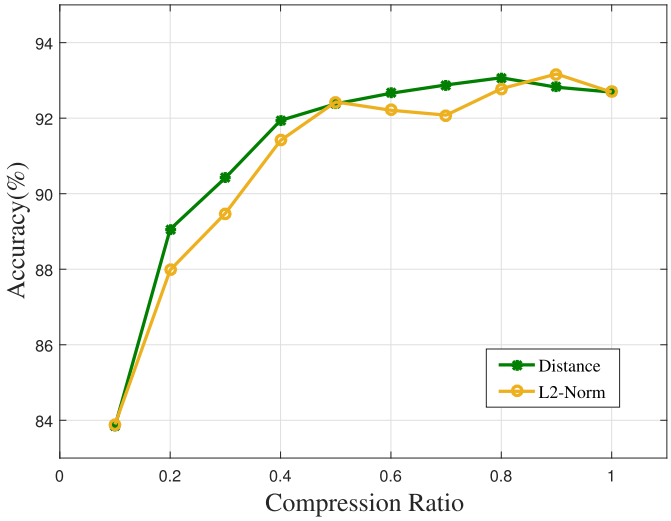


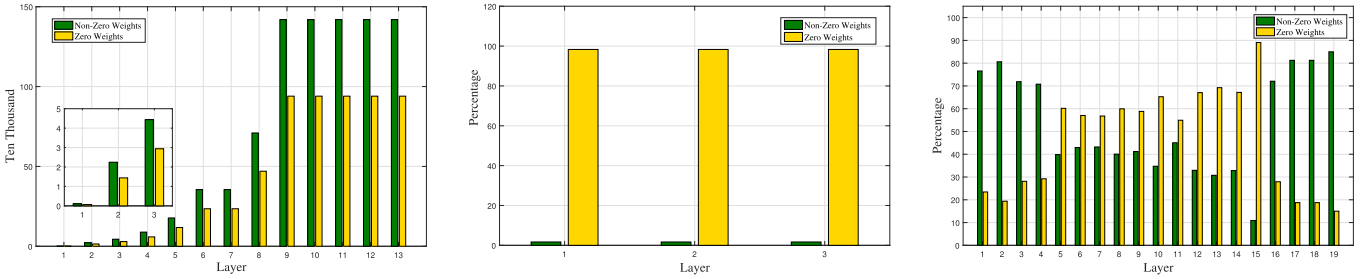
Fig. 8. Accuracy of VGG16 with different compression ratio.

3) FPGA: The proposed accelerator is evaluated on the Xilinx ZCU102 platform, as shown in Fig. 7. The ZCU102 is a general purpose evaluation board based on the Zynq UltraScale+ XCZU9EG. Zynq series is a kind of SoC, combines a multicore ARM CPU as PS and a hardware programmable FPGA as PL. Totally 4GB + 512MB DDR4 is provided onboard for data buffer and 32.1Mb on-chip memory for cache the data being processed [41]. Peta-Linux, a simplified version Linux system designed by Xilinx for Zynq Series, is used as the execution system of the architecture. Power monitor is used to measure the power of the whole board since measuring the power of the Zynq chip alone is unattainable.

B. Experimental Results

1) Model Pruning Results: This paper verifies the proposed mixed pruning method on two different models: VGG-16 for classification and FCN8s for segmentation. These two models contain all of the modules in the framework presented in this paper and can be used to test the performance of individual modules. The VGG16 and FCN8s used in this paper are both original versions and are trained on 8 pieces of 2080Ti.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.



(a) Filter Pruning Results of VGG16 (b) Weight Pruning Results of FC Layers (c) Filter Pruning Results of FCN8s

Fig. 9. Pruning results of both CNN model.

The VGG16 is trained on ImageNet dataset for and FCN8s is trained on Cityscapes dataset. The VGG16 is trained for 100 epochs by SGD optimizer with learning rate 0.1 during filter pruning and 80 epochs by SGD optimizer with learning rate 0.01 during weight pruning. Note that the weight pruning is trained after filter pruning and with the convolution layer parameters fixed. The FCN8s is trained for 500 epochs with learning rate 1×10^{-4}. The specific experimental conclusions are as follows.

Primarily, the effects of different criteria on filter pruning results are evaluated. L2-Normalization and distance parameters are used to test the effect of pruning on accuracy. Normalization is an effective method to improve the learning rate. Distance parameter is the Euclidean distance between a filter and the geometric median of the layer that filter belongs to Fig. 8 shows the influence of the compression ratio on accuracy. For VGG16, the trends of both criteria are consistent. Comparing the two different criteria, the accuracy difference between them is within 0.4% when the compression ratio is larger than 0.5, and using distance parameter tends to have better results.

Secondly, the performance under different compression ratio is analyzed. In this paper, compression ratio means the ratio of model size after compression to that before compression. As shown in Fig. 8, the number of parameters does have a great impact on the accuracy of a certain model. Nevertheless, the accuracy only decreases significantly when the ratio is less than 0.5, which means almost half the parameters can be removed.

Eventually, under the appropriate pruning method and compression ratio, the mixed pruning achieves good performance with a fully compressed model. Filter pruning is applied to the convolution layers and deconvolution layers of VGG16 and FCN8s, and weight pruning is applied to the FC layers. The pruning results are shown in Fig. 9 and Table II. Pruning reduces computation complexity while reducing model size by eliminating redundant neurons and connections. More importantly, the performance of the model has barely changed, as shown in Table II. For VGG16, 86.8% of parameters are pruned with only 0.52% accuracy loss with 31% drop in operations. For FCN8s, after pruning 55.65% filters, there is a 0.29% improvement in MeanIoU and the operations decreased by 47.20%. The experimental results show that proper compression of the model can improve the performance in the allowed error range. The comparison of different compression methods on VGG16 is shown in Table III. Of the above methods, the mixed pruning method performs well in both compression ratio and accuracy.

TABLE II
PRUNING RESULTS OF VGG16 AND FCN8S

CNN Model	VGG16	Pruned VGG16	FCN8s	Pruned FCN8s
Parameters (Million)	37.69	**10.02**	18.64	**6.84**
Filters	5216	**3475**	5236	**2322**
Operation (Gops)	1.22	**0.838**	43.03	**22.72**
Accuracy(%)	92.69	92.17	-	-
MeanIoU(%)	-	-	77.44	**78.73**

TABLE III
COMPARISON OF DIFFERENT MODEL COMPRESSION METHODS ON VGG16

Method	Accuracy(%)	Pruned Parameters(%)
Efficient ConvNets	-0.13	64
APoZ	+3	61.4
Taylor	-3	40
Ours	-0.52	86.8

TABLE IV
DETAILED UTILIZATION OF HARDWARE RESOURCES

	BRAM	DSP48E	FIFO	LUT
VGG16-16bit	1573	655	52088	78236
VGG16-8bit	851	654	58024	71992
FCN8s-16bit	1184	416	60342	66561
FCN8s-8bit	742	698	71468	78349

2) Hardware Performance: This paper tests two designs with different bit-width of the proposed architecture, trying to find a balance between accuracy, bandwidth and power consumption. The resource occupation of both designs is shown in Table IV. Benefiting from the noise immunity of neural networks, the accuracy loss brought by reducing bit-width is minimal. Therefore, resource utilization and latency are the key factors to consider. The results show that the resource utilization has little to do with bit-width except BRAM. Changing the data from 8-bit to 16-bit results the BRAMs increased by 84.4% and 59.57% for VGG16 and FCN8s. Higher bit width will take up more BRAMs, and less data are transferred over one clock period, leading to higher delays. The maximum running frequency of 16-bit can not exceed 200MHz, where 8-bit design can reach 300MHz, which is because low bit-width data can reduce the number of combinational logic, and can

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PERFORMANCE COMPARISON AMONG CPU, GPU, AND FPGA

CNN Model	VGG16				FCN8s			
Platform	i7 7700	RTX2080Ti	ZCU102	ZCU102	i7 7700	RTX2080Ti	ZCU102	ZCU102
Clocks(MHz)	3600	1500	200	300	3600	1500	200	300
Precision	Float-32	Float-32	Fixed-16	Fixed-8	Float-32	Float-32	Fixed-16	Fixed-8
Latency(ms/frame)	930.08	14.74	101.326	73.848	1211.7	92.69	165.65	142.41
Performance(GOP/s)	36.66	2313.43	231.92	318.22	35.46	1314.47	135.77	157.92
Power(W)	67	293	24.3	23.7	67	282	25.1	24.5
Energy Efficiency(GOP/s/W)	0.547	7.90	9.544	**13.43**	0.53	4.66	5.41	**6.52**

achieve a more efficient pipeline under the condition of limited hardware resources. Furthermore, using more BRAMs will result in higher power consumption, which affects the final performance of the design.

The memory reused method and modified CSC format optimize the occupancy of BRAMs, and enable large models like FCN8s to be implemented on FPGA with improved execution efficiency. The FIFO resources are heavily used for the row tile method to improve the access efficiency of feature maps under limited bandwidth. In spite of simultaneous parallel computation of CPEs has high demand for DSP resources, it improves the computation speed of the convolution layer. In general, the proposed architecture achieves better performance while optimizing resource allocation on-chip, which is a feasible solution for deployment of embedded CNN.

Several CNN models are deployed on different platforms, including ZCU102 FPGA, Nvidia 2080Ti GPU and intel i7 7700 CPU. The performance of the three devices is shown in Table V. The following statements are made before analyzing the results: i. The measured performance in Table V is the average performance since it is impossible to maintain the theoretical maximum performance in runtime. ii. The difference between the running power and standby power of GPU and CPU is approximately equal to the actual power of the device, but the power of ZCU102 is only increased by about 1W at runtime. iii. The original model is executed on the GPU and CPU, while the pruned model is executed on ZCU102 using the accelerator proposed in this paper. iv. The batch size set for all three platforms is 1, since large batch size will increase latency for real-time applications even if it makes full use of the parallelism of GPU.

As shown in Table V, GPU has incomparable advantage in processing speed. For VGG16, the processing speed of GPU can be 5× faster than FPGA and 63× faster than CPU. However, lower latency is built on higher power dissipation, which is one of the reasons why GPU is unsuitable for embedded systems. The power dissipation of FPGA is only 8% of the GPU and 35% of the CPU, and the energy efficiency is 1.7× that of GPU and over 10× of CPU. Meanwhile, thanks to the substantial compression of the model by pruning, the latency of FPGA is low enough to meet the needs of practical applications.

The performance compared with other frameworks is shown in Table VI. The fpgaConvNet designs an automatic framework to optimize the deployment of CNN on FPGA and achieves high performance density, but it performs poorly for network with a large number of parameters. Tight interaction between software and hardware makes NEURAghe to take full use of bandwidth, which is also a bottleneck that limits its performance. The most effective way to accelerate the inference of CNN is to reduce operand. The DNNVM adopts the operation fusion and CONNA uses pruning and quantization to reduce operand. The framework proposed in this paper uses model compression and quantization, and a special acceleration architecture is designed for the sparse model after compression. Therefore, the method proposed in this paper is more radical and the performance is better than the methods previously proposed in some aspects.

TABLE VI
PERFORMANCE COMPARISON OF DIFFERENT FRAMEWORKS ON VGG16

	[42]	[43]	[44]	[25]	Ours
Platform	Zynq7045	Zynq7045	ZCU102	ZCU102	ZCU102
Clock(MHz)	125	125	330	200	300
Latency(ms)	249.5	181.205	91.89	102.77	73.848
GOP/s	155.81	123	334	301.17	318.22
GOP/s/W	38.95	12.3	123.7	13.69	13.43

V. CONCLUSION

This paper proposes a complete framework to deploy CNN on embedded systems. A mixed pruning method is used to compress the model with almost no loss of accuracy. An accelerator based on FPGA is proposed for embedded systems. Two different models, VGG16 for classification and FCN8s for segmentation, have been deployed. The performance of three different platforms has also been tested, and the energy efficiency of the architecture proposed in this paper is up to 1.7× better than GPU and over 20× better than CPU. More importantly, the advantages of the pruned model are fully used to improve hardware resource utilization, memory access efficiency and computation efficiency.

The framework proposed in this paper provides a solution for the deployment of neural networks in intelligent vehicles, drones and other fields. This method will be tried to be applied to the environment perception of intelligent vehicles to improve the real-time performance of the perception algorithm in future works.

REFERENCES

[1] Y. Zhu, W. X. Zheng, and D. Zhou, "Quasi-synchronization of discrete-time Lur'e-type switched systems with parameter mismatches and relaxed PDT constraints," *IEEE Trans. Cybern.*, vol. 50, no. 5, pp. 2026–2037, May 2020.

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*. [Online]. Available: http://arxiv.org/abs/1409.1556

[3] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.

[4] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.

[5] W. Sun, J. Zhang, and Z. Liu, "Two-time-scale redesign for antilock braking systems of ground vehicles," *IEEE Trans. Ind. Electron.*, vol. 66, no. 6, pp. 4577–4586, Jun. 2019.

[6] W. Sun, X. Wang, and C. Zhang, "A model-free control strategy for vehicle lateral stability with adaptive dynamic programming," *IEEE Trans. Ind. Electron.*, vol. 67, no. 12, pp. 10693–10701, Dec. 2020.

[7] X. Zhao, X. Wang, S. Zhang, and G. Zong, "Adaptive neural back-stepping control design for a class of nonsmooth nonlinear systems," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 49, no. 9, pp. 1820–1831, Sep. 2019.

[8] P. G. Mousouliotis and L. P. Petrou, "Software-defined FPGA accelerator design for mobile deep learning applications," in *Proc. Int. Symp. Appl. Reconfigurable Comput.* Cham, Switzerland: Springer, 2019, pp. 68–77.

[9] C. Zhang, D. Wu, J. Sun, G. Sun, G. Luo, and J. Cong, "Energy-efficient CNN implementation on a deeply pipelined FPGA cluster," in *Proc. Int. Symp. Low Power Electron. Des.*, Aug. 2016, pp. 326–331.

[10] A. Jafari, A. Ganesan, C. S. K. Thalisetty, V. Sivasubramanian, T. Oates, and T. Mohsenin, "SensorNet: A scalable and low-power deep convolutional neural network for multimodal data classification," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 1, pp. 274–287, Jan. 2019.

[11] H. Pan, X. Jing, W. Sun, and H. Gao, "A bioinspired dynamics-based adaptive tracking control for nonlinear suspension systems," *IEEE Trans. Control Syst. Technol.*, vol. 26, no. 3, pp. 903–914, May 2018.

[12] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-based accelerators of deep learning networks for learning and classification: A review," *IEEE Access*, vol. 7, pp. 7823–7859, 2019.

[13] H. Pan, X. Jing, W. Sun, and Z. Li, "Analysis and design of a bioinspired vibration sensor system in noisy environment," *IEEE/ASME Trans. Mechtron.*, vol. 23, no. 2, pp. 845–855, Apr. 2018.

[14] C. Yang, Y. Wang, X. Wang, and L. Geng, "A stride-based convolution decomposition method to stretch CNN acceleration algorithms for efficient and flexible hardware implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 9, pp. 3007–3020, Sep. 2020.

[15] T. Guan, P. Liu, X. Zeng, M. Kim, and M. Seok, "Recursive binary neural network training model for efficient usage of on-chip memory," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 7, pp. 2593–2605, Jul. 2019.

[16] J. Wang, J. Lin, and Z. Wang, "Efficient hardware architectures for deep convolutional neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 6, pp. 1941–1953, Jun. 2018.

[17] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*. [Online]. Available: http://arxiv.org/abs/1510.00149

[18] Z. Chen, Z. Chen, J. Lin, S. Liu, and W. Li, "Deep neural network acceleration based on low-rank approximated channel pruning," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 4, pp. 1232–1244, Apr. 2020.

[19] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4340–4349.

[20] Y. He, G. Kang, X. Dong, Y. Fu, and Y. Yang, "Soft filter pruning for accelerating deep convolutional neural networks," 2018, *arXiv:1808.06866*. [Online]. Available: http://arxiv.org/abs/1808.06866

[21] Y. Lyu, L. Bai, and X. Huang, "ChipNet: Real-time LiDAR processing for drivable region segmentation on an FPGA," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 5, pp. 1769–1779, May 2019.

[22] H. Pan and W. Sun, "Nonlinear output feedback finite-time control for vehicle active suspension systems," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2073–2082, Apr. 2019.

[23] L. Zhang, Y. Zhu, and W. X. Zheng, "Synchronization and state estimation of a class of hierarchical hybrid neural networks with time-varying delays," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 2, pp. 459–470, Feb. 2016.

[24] X. Zhao, X. Wang, G. Zong, and X. Zheng, "Adaptive neural tracking control for switched high-order stochastic nonlinear systems," *IEEE Trans. Cybern.*, vol. 47, no. 10, pp. 3088–3099, Oct. 2017.

[25] R. Struharik, B. Vukobratović, A. Erdeljan, and D. Rakanović, "CoNNA–compressed CNN hardware accelerator," in *Proc. 21st Euromicro Conf. Digit. Syst. Design (DSD)*, Aug. 2018, pp. 365–372.

[26] K. Guo *et al.*, "Angel-eye: A complete design flow for mapping CNN onto embedded FPGA," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 1, pp. 35–47, Jan. 2018.

[27] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2015, pp. 161–170.

[28] Y.-J. Lin and T. S. Chang, "Data and hardware efficient design for convolutional neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 5, pp. 1642–1651, May 2018.

[29] J.-H. Luo, J. Wu, and W. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 5058–5066.

[30] H. Pan, H. Li, W. Sun, and Z. Wang, "Adaptive fault-tolerant compensation control and its application to nonlinear suspension systems," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 50, no. 5, pp. 1766–1776, May 2020.

[31] Q. Huang, K. Zhou, S. You, and U. Neumann, "Learning to prune filters in convolutional neural networks," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Mar. 2018, pp. 709–718.

[32] S. Han *et al.*, "EIE: Efficient inference engine on compressed deep neural network," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 243–254.

[33] J. Yu, A. Lukefahr, D. Palframan, G. Dasika, R. Das, and S. Mahlke, "Scalpel: Customizing DNN pruning to the underlying hardware parallelism," *ACM SIGARCH Comput. Archit. News*, vol. 45, no. 2, pp. 548–560, Sep. 2017.

[34] W. Yang, L. Jin, S. Wang, Z. Cu, X. Chen, and L. Chen, "Thinning of convolutional neural network with mixed pruning," *IET Image Process.*, vol. 13, no. 5, pp. 779–784, Apr. 2019.

[35] L. Zhang, Y. Zhu, and W. X. Zheng, "State estimation of discrete-time switched neural networks with multiple communication channels," *IEEE Trans. Cybern.*, vol. 47, no. 4, pp. 1028–1040, Apr. 2017.

[36] A. Buluç, J. T. Fineman, M. Frigo, J. R. Gilbert, and C. E. Leiserson, "Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks," in *Proc. 21st Annu. Symp. Parallelism Algorithms Archit. (SPAA)*, 2009, pp. 233–244.

[37] M. Zhang, L. Li, H. Wang, Y. Liu, H. Qin, and W. Zhao, "Optimized compression for implementing convolutional neural networks on FPGA," *Electronics*, vol. 8, no. 3, p. 295, Mar. 2019.

[38] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, "FP-BNN: Binarized neural network on FPGA," *Neurocomputing*, vol. 275, pp. 1072–1086, Jan. 2018.

[39] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.

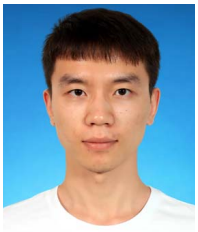
[40] S. Liu, H. Fan, X. Niu, H.-C. Ng, Y. Chu, and W. Luk, "Optimizing CNN-based segmentation with deeply customized convolutional and deconvolutional architectures on FPGA," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 11, no. 3, p. 19, Dec. 2018.

[41] Xilinx's Zynq UltraScale. (Aug. 23, 2018). *MPSoC ZCU102 Evaluation Kit*. [Online]. Available: https://www.xilinx.com/products/boards-andkits/ek-u1-zcu102-g.html

[42] S. I. Venieris and C.-S. Bouganis, "FpgaConvNet: Mapping regular and irregular convolutional neural networks on FPGAs," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 2, pp. 326–342, Feb. 2019.

[43] P. Meloni *et al.*, "NEURAghe: Exploiting CPU-FPGA synergies for efficient and flexible CNN inference acceleration on Zynq SoCs," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 11, no. 3, pp. 1–24, Dec. 2018.

[44] Y. Xing *et al.*, "DNNVM: End-to-end compiler leveraging heterogeneous optimizations on FPGA-based CNN accelerators," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2668–2681, Oct. 2020.



Xuepeng Chang received the bachelor's degree in engineering from the Harbin Institute of Technology, Harbin, China, in 2018, where he is currently pursuing the Ph.D. degree with the Research Institute of Intelligent Control and Systems. His research interests include deep learning, computer vision, and autonomous vehicle environment perception.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.



Huihui Pan received the Ph.D. degree in control science and engineering from the Harbin Institute of Technology, Harbin, China, in 2017, and the Ph.D. degree in mechanical engineering from The Hong Kong Polytechnic University, Hong Kong, in 2018.

Since December 2017, he has been with the Research Institute of Intelligent Control and Systems, Harbin Institute of Technology. His research interests include nonlinear control, vehicle dynamic control, and intelligent vehicles.



Weiyang Lin received the bachelor's and M.Sc. degrees in mechanical engineering from the Harbin Institute of Technology, China, in 2006 and 2008, respectively, and the Ph.D. degree in mechatronics engineering from the Shenzhen Graduate School, Harbin Institute of Technology, in 2014.

He is currently an Associate Professor with the Research Institute of Intelligent Control and Systems, Harbin Institute of Technology. His research interests include parallel manipulators, robotic motion control, and medical robotic design and control.



Huijun Gao (Fellow, IEEE) received the Ph.D. degree in control science and engineering from the Harbin Institute of Technology, Harbin, China, in 2005.

From 2005 to 2007, he carried out his Post-Doctoral Research with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB, Canada. Since 2004, he has been with the Harbin Institute of Technology, where he is currently a Full Professor, the Director of the Research Institute of Intelligent Control and Systems, and the Director of the Interdisciplinary Research Center. His research interests include intelligent and robust control, robotics, mechatronics, and their engineering applications. He is an IEEE Industrial Electronics Society Vice President of Workshop Activities and a Council Member of the International Federation of Automatic Control (IFAC). He also serves as the Co-Editor-in-Chief for the IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, a Senior Editor for the IEEE/ASME TRANSACTIONS ON MECHATRONICS, and an Associate Editor for *Automatica*, the IEEE TRANSACTIONS ON CYBERNETICS, and the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on June 01,2026 at 08:59:44 UTC from IEEE Xplore. Restrictions apply.